An Integrated ROS 2 Ackermann Vehicle Stack for Scaled Autonomous Navigation: Simulation, Planning, Control, Localization, and Embedded Actuation

Andrew Abdelmalak
Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
andrew.abdelmalak@student.guc.edu.eg

Daniel Boules
Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
daniel.boules@student.guc.edu.eg

David Girgis
Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
david.girgis@student.guc.edu.eg

Kirolous Kirolous
Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
kirolous.kirolous@student.guc.edu.eg

Samir Yacoub
Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
samir.yacoub@student.guc.edu.eg

Youssef Salama
Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
youssef.salama@student.guc.edu.eg

***Abstract*—Many scaled-vehicle projects stop at ROS bring-up, teleoperation, or isolated localization experiments, leaving the full autonomy pipeline only partially validated. This paper presents the final integrated stack we built for a scaled Ackermann vehicle using ROS 2 Jazzy, Gazebo Harmonic, Python planning and control nodes, a Raspberry Pi 4 host, and a Raspberry Pi Pico-based embedded actuation layer. The final system connects three structured track scenarios to a shared planner, a feedforward-plus-PID longitudinal controller, a Stanley-style lateral controller, and a linearized Kalman-filter localization node. In the exported Milestone 5 simulation artifacts, with injected measurement noise of 0.03 m in position, 0.04 rad in heading, 0.05 m/s in speed, and 0.08 rad/s in yaw rate, the filter reduces position root-mean-square error from 4.17 cm to 0.71 cm on the straight track and to 0.69 cm on the racing track, while heading root-mean-square error drops from 2.19° to 0.42° in both cases. Hardware demonstrations confirm that the same high-level pipeline can run headlessly on the physical platform for predefined straight, obstacle-avoidance, and city-style tracks. The main limitation is that the system is validated on known tracks with proprioceptive sensing only, not in generalized obstacle-rich environments.**

***Index Terms*—ROS 2, Ackermann steering, autonomous vehicle, path planning, Kalman filtering**

I. Introduction

Scaled autonomous vehicles are useful precisely because they expose the same integration problem faced by larger platforms: middleware, control, planning, state estimation, and embedded actuation must share a consistent command-state interface before autonomy claims are meaningful. For Ackermann vehicles, that integration is further constrained by nonholonomic steering geometry, limited sensing, and the dynamics mismatch between simulator-friendly models and small physical chassis [1]–[3].

The literature covers each major subsystem independently. ROS 2 and Gazebo provide the middleware and simulation backbone for modular robotic systems [1], [2]. Stanley-based lateral control remains a practical choice for Ackermann path tracking because it combines heading and cross-track correction in a compact law [4]. Kalman-based sensor fusion is also well established for vehicle localization when encoder and inertial measurements must be combined [5], [6]. The gap, however, is not algorithm availability; it is showing a single low-cost stack that carries shared interfaces from simulation bring-up to planning, control, embedded actuation, and localization demonstrations on one platform.

This paper presents our final autonomous systems stack as an integrated autonomy pipeline rather than a milestone checklist. The development process was milestone-based, but the final contribution is the connected stack shown in Fig. 1. The paper makes four project-level contributions:

1) it documents a unified ROS 2 Jazzy Ackermann stack spanning Gazebo simulation, shared planning and control nodes, Raspberry Pi host-side execution, and Pico-based embedded actuation and sensing;
2) it shows that the final value lies in the end-to-end autonomy pipeline, not ROS bring-up alone, by connecting validation, planning, control, localization, and hardware demonstrations into one consistent workflow;
3) it provides source-backed quantitative evidence from the exported MS5 localization artifacts, including up to 83.5% position-RMSE reduction and about 81% heading-RMSE reduction under the documented noise settings;

4) it states the observed limitation directly: the platform demonstrates autonomy on predefined tracks with limited sensing, not generalized autonomy in dynamic, obstacle-rich scenes.

The remainder of the paper is organized as follows. Section II summarizes the most relevant technical context. Section III presents the integrated system architecture. Section IV describes the vehicle platform and experimental setup. Section V covers planning, control, and localization. Section VI reports the results. Section VII interprets those results and states the limitations. Section VIII concludes the paper.

II. Related Work

ROS 2 is a natural fit for staged robotics development because the DDS-based publish-subscribe model decouples sensing, planning, control, and hardware interfaces into reusable nodes [1]. Gazebo Harmonic complements that workflow by providing simulated sensors, actuators, and vehicle worlds that can be exercised before hardware deployment [2].

For Ackermann steering, the essential geometric relationship remains the shared instantaneous center of rotation across the steered wheels, as analyzed for multi-wheeled obstacle-crossing platforms in [3]. This is commonly approximated with the kinematic bicycle model, which is accurate enough for low- to moderate-speed scaled platforms and remains a useful basis for controller design. Stanley tracking is still attractive in this setting because heading error is handled explicitly instead of being hidden inside a lookahead heuristic [4].

Sensor fusion for mobile robots is also well established. Combining wheel encoders with inertial sensing via Kalman filtering improves dead-reckoning consistency [5], while multi-sensor extended Kalman filtering mitigates drift in structured urban environments [6]. The present work does not propose a new planner or estimator; it shows how these known components can be assembled into a coherent ROS 2 autonomy stack and evaluated with honest scope boundaries.

III. Integrated System Architecture

Figure 1 summarizes the final stack. The key design decision is that simulation and hardware share the same high-level planner, longitudinal controller, lateral controller, and filtered odometry interface. Gazebo publishes ideal odometry into the localization node during simulation, while the physical platform supplies encoder and IMU telemetry through the Pico. The planner consumes filtered odometry and publishes desired speed, lane, and heading targets. The speed controller converts the desired speed into a bounded longitudinal command, and the Stanley-style lateral controller converts the lane and heading targets into steering action. The command path then terminates either at Gazebo through `ros_gz_bridge` or at the physical vehicle through a USB serial link.

IV. Vehicle Platform and Experimental Setup

A. Vehicle Platform

The physical platform uses a Raspberry Pi 4 running Ubuntu Noble 24.04 and ROS 2 Jazzy as the host computer, paired with an embedded actuator and sensing controller. In Milestones 2–3, an Arduino Uno (ATmega328P) closed a 50 Hz feedforward-plus-proportional wheel-speed loop (748 ticks/rev, 3,663 ticks/m). For Milestones 4–5, we migrated to a Raspberry Pi Pico RP2040 that drives the L298N motor stage and MG995 steering servo, reads the quadrature encoder (7,000 ticks/m) and MPU-6050 IMU (300-sample startup bias calibration), and exchanges `SPD`/`STR` commands and `VEL`/`THETA`/`TICKS` telemetry with the Pi over USB serial at 115200 baud with a 2000 ms timeout failsafe. When commanded steering is near zero, the Pico engages an onboard PD heading-hold loop ($K_p = 1.5$, $K_d = 0.8$) using the calibrated gyro rate. The final integrated simulation configuration uses a hardware-scale wheelbase of 0.255 m so that the Gazebo controller and planner operate on vehicle geometry closer to the physical chassis.

B. Validation Baseline

Before autonomy behaviors were evaluated, the base software path was verified on both the host-side Pi and the simulation workstation. The Pi-side `ros2 doctor` run passed all five checks, and custom validation nodes confirmed sustained ROS 2 execution on both platforms. The Gazebo teleoperation mode then confirmed that command publication, vehicle motion, and feedback logging were all connected before the autonomy-specific nodes were introduced. Figure 3 condenses those bring-up artifacts into one panel because their role in the final project is evidentiary, not narrative.

C. Track Scenarios and Final Parameters

The final autonomy stack was exercised on three predefined scenarios: a straight track, a two-lane racing track with fixed obstacles, and a city-style track with repeated turns. Table I lists the runtime parameters that matter most for interpreting the results.

TABLE I
Key Runtime Parameters in the Final Integrated Stack

Group	Item	Value
Vehicle	Wheelbase	0.255 m
Vehicle	Pico control loop	50 Hz
Planner	Straight-track speed	0.50 m/s
Planner	Racing straight / lane-change speed	0.26 / 0.14 m/s
Planner	City straight / turn speed	0.35 / 0.18 m/s
Control	Speed PID	0.85, 0.10, 0.03
Control	Stanley / heading	0.30; 0.50 / 0.15
Loc.	Meas. std $(x, y, \theta, v, \omega)$	0.03, 0.03, 0.04, 0.05, 0.08
Loc.	Proc. std (v, ω)	0.025, 0.05

V. Planning, Control, and Localization

A. Longitudinal and Lateral Control

The longitudinal controller is a feedforward-plus-PID loop that tracks the planner's desired speed $v_{\text{ref}}(t)$ with integral anti-windup (± 1.5 m/s) and acceleration slew-rate limiting ($a_{\max} = 0.30$ m/s^2):

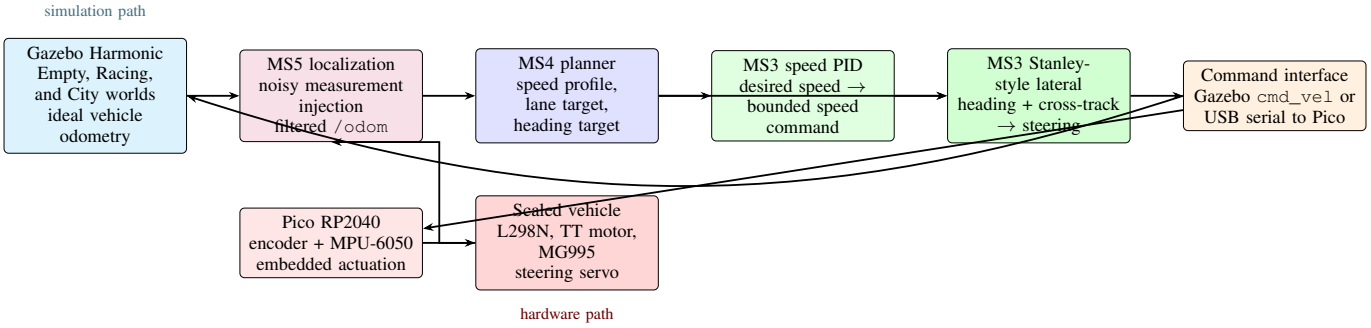


Fig. 1. Final integrated autonomy stack shared by the Gazebo validation path and the physical vehicle path.



Pi 4 host
Pico RP2040
Drive L298N + TT motor
Steering MG995 servo
Sensing encoder + MPU-6050
Loop 50 Hz

Fig. 2. Scaled vehicle platform used for the hardware demonstrations. The Pi hosts the ROS 2 autonomy nodes, while the Pico closes the embedded actuation and sensing loop.

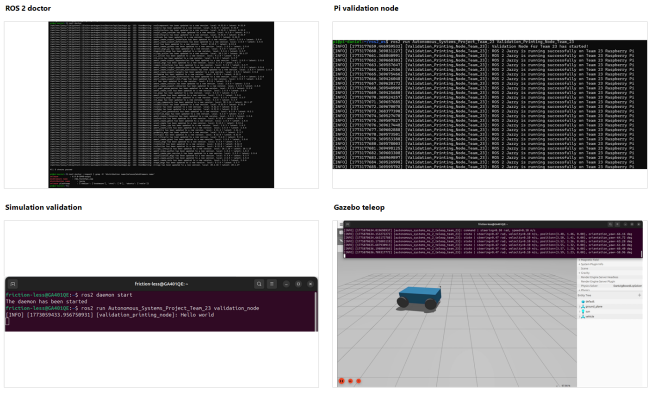


Fig. 3. Condensed bring-up evidence: ROS 2 diagnostics on the Raspberry Pi, hardware and simulation validation nodes, and Gazebo teleoperation for the initial motion and feedback path.

$$u(t) = v_{\text{ref}}(t) + K_p e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}, \quad (1)$$

where $e(t) = v_{\text{ref}}(t) - v(t)$ is the speed error. The lateral controller is best described as a Stanley-style controller rather than a textbook Stanley implementation because the shipped node combines heading error, a Stanley cross-track term, integral bias correction, and yaw-rate damping:

$$\delta = K_\psi e_\psi + \arctan\left(\frac{k_y e_y}{\max(|v|, v_0)}\right) + K_I \int e_y\, dt - K_d \dot{\psi}, \quad (2)$$

where e_ψ is heading error, e_y is cross-track error, and $v_0 = \max(\texttt{lateral_kd}, 0.05)$ (0.10 m/s in the MS4/MS5 launch configuration, with $K_I = 0.0$ in MS4/MS5 and 0.02 in MS3) softens the low-speed response. This form matches the node that was ultimately launched in the final stack and is more precise than calling the implementation a pure Stanley controller.

B. Track Planner

The planner publishes desired speed, desired lane, and desired heading targets. On the straight track, it mainly acts as a centerline and stop-state generator. On the racing track, it applies a geometry-aware lane profile that starts in the left lane, transitions right early enough to clear a fixed obstacle, and then returns left for the second obstacle. The exported racing-track evaluation uses a scheduled speed reduction from 0.26 m/s on straights to 0.14 m/s during lane changes. On the city track, the MS5 launch profile reduces the target speed from 0.35 m/s on straights to 0.18 m/s in turns (compared with the planner node defaults of 0.45 m/s and 0.25 m/s used in MS4).

C. Kalman-Filter Localization

The final localization node operates on the state $\mathbf{x} = [x, y, \theta, v, \omega]^\top$. During simulation-side evaluation, it subscribes to Gazebo's ideal odometry, injects white Gaussian noise using the standard deviations listed in Table I, and then applies a linearized discrete Kalman filter. The motion model retains the Ackermann-inspired kinematic structure through the yaw-rate relationship

$$\dot{\psi} = \frac{v \tan \delta}{L}, \tag{3}$$

while the filter predicts and corrects the state with the standard linear Kalman recursion. The value of this node in the final project is twofold: it provides a filtered `/odom` topic that the planner and controllers can share, and it also exports the CSV and PNG artifacts used for the quantitative evaluation in Section VI.

VI. Results

A. From Bring-Up to Integrated Execution

The validation evidence in Fig. 3 confirms that the project foundation is real rather than implied: the Raspberry Pi installation passes all five `ros2 doctor` checks, custom validation nodes run on both platforms, and Gazebo teleoperation closes the early command-feedback loop. This baseline matters because the later autonomy behaviors are only meaningful once that communication path is stable.

B. Straight-Track State Tracking Under Injected Noise

Figure 4 shows the exported straight-track MS5 artifact over a 10.34 s run. In this scenario, the noisy state has a position RMSE of 4.17 cm relative to the ideal trajectory, while the filtered estimate reduces that error to 0.71 cm. Heading RMSE drops from 2.19° to 0.42°, and speed RMSE drops from 0.0495 m/s to 0.0243 m/s. The key point is not only that the filter smooths the curves, but that the straight track keeps model mismatch low enough for the prediction step to meaningfully regularize each noisy update.

The filtered lateral deviation on the straight track is 0.0034 m RMS, with a 0.0218 m peak absolute deviation relative to the ideal centerline. That small lateral error is consistent with the simple geometry of the track and explains why the stack is able to use the straight-track scenario as a sanity check for the full pipeline before more aggressive maneuvering.

C. Racing-Track Path Following and Localization

The racing-track artifact is more revealing because it exercises both lane changes and localization simultaneously. The planner lowers the target speed to 0.14 m/s during lane transitions and restores 0.26 m/s on straights. The result should be interpreted as path-following performance under that scheduled profile, not as an isolated ablation proving that the speed reduction alone caused the improvement.

Figure 5 shows the corresponding XY path. The filtered path remains close to the ideal lane-change corridor while the noisy path spreads more widely around it. Quantitatively, the racing scenario improves from 4.17 cm to 0.69 cm position RMSE and from 2.19° to 0.42° heading RMSE. Filtered lateral error drops to 0.0026 m RMS. Because the same noise configuration and filter structure are used on both the straight and racing cases, the near-identical RMSE reduction indicates that the localization benefit is robust across at least two distinct path geometries, not only one special-case run.

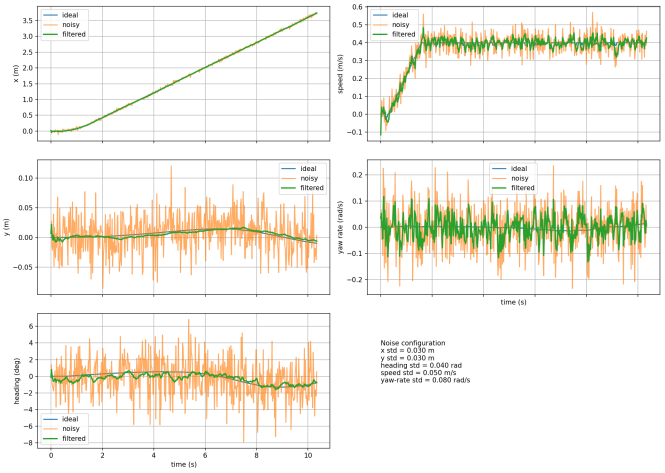


Fig. 4. Exported MS5 straight-track artifact. The filtered states follow the ideal x, y, heading, speed, and yaw-rate histories much more closely than the noisy measurements over the 10.34 s run.

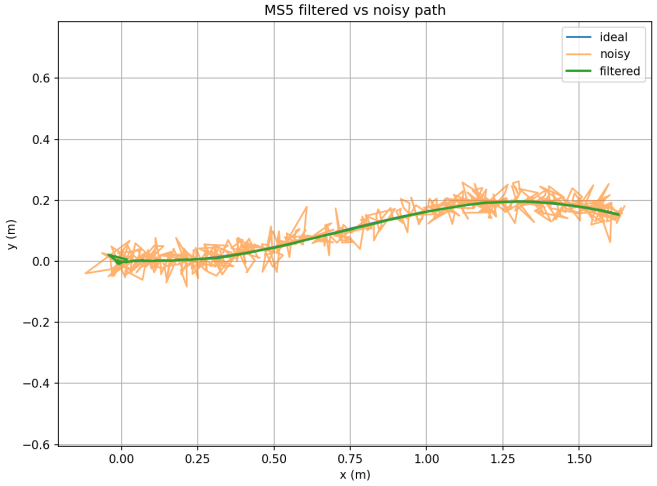


Fig. 5. Exported MS5 racing-track XY path. The filtered trajectory stays near the ideal lane-change corridor through the obstacle-avoidance sequence, while the noisy path diverges more visibly.

Table II and Fig. 6 summarize the source-backed quantitative gains that can be claimed from the available course artifacts.

TABLE II
MS5 Localization Improvements from Exported CSV Artifacts

Scenario	Pos. gain	Heading gain	Speed gain
Straight track	82.9%	81.0%	51.0%
Racing track	83.5%	80.7%	47.2%

D. Final Integrated Autonomy Demonstrations

The available hardware evidence is qualitative rather than numeric, but it still matters because it validates the final system boundary claimed in this work. The hardware run in

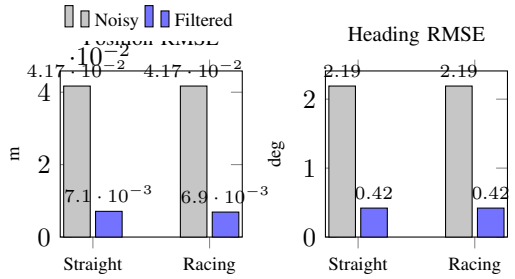


Fig. 6. Absolute RMSE values from the exported MS5 artifacts. The Kalman filter consistently reduces both position and heading error across the two evaluated scenarios.

Fig. 7 shows that the onboard Pi and Pico stack can execute a representative predefined autonomy demonstration without a GUI-driven operator. The paired MS5 node graph confirms that localization, planning, and control were composed as one ROS 2 pipeline rather than as disconnected milestone scripts.

What cannot be claimed from the available hardware artifacts is scenario-by- scenario hardware benchmarking or real-world RMSE, because the course folder contains images and videos for the demonstrations but not synchronized hardware log files with ground truth. The final report therefore treats hardware autonomy as demonstrated qualitatively, while the quantitative comparisons in this paper remain tied to the exported MS5 simulation artifacts.

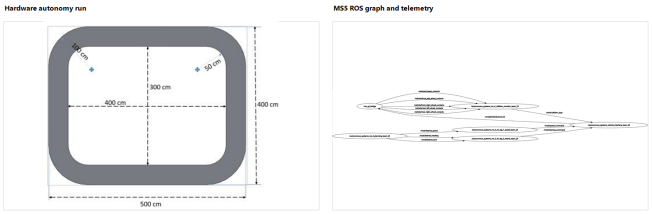


Fig. 7. Final integrated autonomy evidence assembled from course artifacts: a physical headless autonomy run and the MS5 ROS 2 node graph / telemetry environment used to validate the full pipeline.

VII. Discussion and Limitations

The main contribution is integration, not any single node. The strongest result of the project is that the same ROS 2 command-state interfaces survive the full trip from Gazebo validation to embedded actuation and back to filtered odometry. This is why the project is more valuable than a ROS installation checklist: the planner, controllers, and localization node are all exercised as one autonomy stack.

The racing-track result should be read as performance under a scheduled planner profile, not as a speed-scheduling ablation. The lane-change speed reduction from 0.26 m/s to 0.14 m/s is part of the documented planner configuration. On a 0.255 m wheelbase vehicle, that slower profile is a plausible design choice, and it is consistent with the clean path shown in Fig. 5. What the available artifacts do not prove in isolation is how much of the improvement is attributable to speed scheduling alone versus the rest of the controller-and-estimator stack.

The localization gains are real but bounded by the evaluation setup. The reported RMSE improvements come from the exported MS5 simulation runs where noise is injected into Gazebo odometry and then filtered. That setup is still useful because it reveals the effect of the estimator under known conditions, but it should not be confused with a hardware-ground-truth benchmark.

The central limitation is track-bounded autonomy with limited sensing. The planner assumes known tracks and fixed obstacle geometry. The hardware stack uses proprioceptive sensing only: encoder and IMU data. As a result, the system does not perceive unexpected obstacles, does not replan around dynamic agents, and does not perform SLAM in unknown environments. A second limitation is that the course artifacts do not include synchronized hardware logs against an external reference, so real-world localization error cannot be quantified honestly from the available material.

VIII. Conclusion

This paper reframed our autonomous systems project as a final integrated Ackermann autonomy stack instead of a milestone chronology. The resulting system links ROS 2 Jazzy, Gazebo Harmonic, shared planning and control nodes, Pico-based embedded actuation, and Kalman-filtered localization across straight, racing, and city-style demonstrations. The strongest quantitative evidence comes from the exported MS5 artifacts, where the localization node reduces position RMSE by about 83% and heading RMSE by about 81% under the documented noise conditions. The strongest practical evidence is that the same stack runs through to a representative headless hardware demonstration. The remaining gap is equally clear: the current project is a known-track autonomy pipeline with limited sensing, not a generalized perception-rich autonomous vehicle.

Acknowledgment

The authors thank Dr. Omar M. Shehata and Eng. Dalia M. Mahfouz for their guidance and support throughout the MCTR 1002 Autonomous Systems course project.

References

[1] Open Robotics, “ROS 2 Jazzy Jalisco Documentation,” https://docs.ros.org/en/jazzy/, 2026, accessed: 2026-06-28.

[2] ——, “Gazebo Harmonic Documentation,” https://gazebosim.org/docs/harmonic/, 2026, accessed: 2026-06-28.

[3] T. Xu, S.-J. Ma, H.-T. Xu, S. Mo, and Y. Li, “Application of Ackermann steering in obstacle crossing platform of six-wheeled robots,” in *Proc. 2nd Int. Symp. Control Eng. Robot. (ISCER)*, 2023, pp. 239–243.

[4] G. M. Hoffmann, C. J. Tomlin, M. Montemerlo, and S. Thrun, “Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing,” in *Proc. Amer. Control Conf. (ACC)*, 2007, pp. 2296–2301.

[5] M. Peiris, H. Lang, and M. El-Gindy, “Kalman filter-based sensor fusion for Ackermann steering mobile robots,” *J. Phys. Conf. Ser.*, vol. 2811, no. 1, p. 012004, Jul. 2024.

[6] A. Georgiev and P. K. Allen, “Localization methods for a mobile robot in urban environments,” *IEEE Trans. Robot.*, vol. 20, no. 5, pp. 851–864, Oct. 2004.